

IDS考试 NumPy / Pandas 常用函数速查表

一、NumPy 基础

创建数组

```
python

import numpy as np

np.array([1, 2, 3])           # 从列表创建1D数组
np.array([[1,2],[3,4]])      # 从嵌套列表创建2D矩阵
np.zeros((n, m))             # n×m全0矩阵（初始化转移矩阵用）
np.ones((n, m))              # n×m全1矩阵
np.eye(n)                    # n×n单位矩阵
np.arange(0, 1.01, 0.01)     # 等差序列（threshold遍历用）
np.linspace(0, 1, 101)       # 等间距序列（和arange效果类似）
```

数组运算

```
python

a + b                        # 逐元素加法
a * b                        # 逐元素乘法
a @ b                        # 矩阵乘法
a.T                          # 转置
a ** 2                       # 逐元素平方

np.sum(a)                    # 所有元素求和
np.sum(a, axis=1)            # 按行求和
np.mean(a)                   # 均值
np.std(a)                    # 标准差
np.sqrt(x)                   # 平方根
np.log(x)                    # 自然对数
np.exp(x)                    # e的x次方
```

布尔运算（分类评估最常用）

```
python
```

```
a == b                # 逐元素比较，返回布尔数组
a >= threshold        # 大于等于threshold
(a >= threshold).astype(int) # 布尔数组转0/1整数数组

np.sum(a == b)        # 统计相等的元素个数
np.sum((a == 1) & (b == 1)) # 同时满足两个条件的元素个数（TP）
```

线性代数

```
python

np.linalg.solve(A, b)          # 解线性方程组 Ax=b（Markov chain用）
np.linalg.svd(X, full_matrices=False) # SVD分解
np.linalg.norm(X, axis=1)      # 按行算欧氏范数（重构误差用）
np.linalg.matrix_power(P, n)   # 矩阵的n次幂（多步转移概率用）
np.linalg.eig(A)               # 特征值分解（求stationary distribution用）
np.linalg.lstsq(A, b)         # 最小二乘法解方程
np.diag(d)                     # 1D数组转对角矩阵
```

随机数

```
python

np.random.choice(n, p=probs) # 按概率分布随机选一个（Markov chain模拟用）
np.random.rand(n)            # n个[0,1)均匀随机数
np.random.randn(n)           # n个标准正态随机数
```

其他常用

```
python

np.argmax(a)              # 返回最大值的索引
np.argmin(a)              # 返回最小值的索引
np.sort(a)                # 排序
np.argsort(a)             # 返回排序后的索引
len(a)                    # 数组长度
a.shape                   # 数组形状（调试shape问题用）
a[:, 1]                   # 取第二列（predict_proba取class 1概率用）
a[:k]                     # 取前k个元素
```

二、Pandas 基础

读取数据

```
python

import pandas as pd

df = pd.read_csv('data/file.csv')    # 读取CSV文件
df.head()                            # 查看前5行
df.shape                             # (行数, 列数)
len(df)                              # 总行数
```

取列/行

```
python

df['Class']                           # 取一列（返回Series）
df['Class'].values                    # 取一列转numpy数组
df['Class'].tolist()                 # 取一列转Python列表
df[['V1', 'V2', 'Amount']]          # 取多列
df[df['Class'] == 1]                 # 筛选满足条件的行
df[df['Class'] == 1].values          # 筛选后转numpy数组
```

常用统计

```
python

df['Class'].unique()                  # 取唯一值（查看有哪些类别）
df['Class'].value_counts()            # 统计每个值出现次数
len(df['Class'].unique())             # 唯一值数量
sorted(df['from'].unique())           # 排序后的唯一值列表
set(df['from'])                       # 去重（和unique类似）
```

遍历

```
python

for _, row in df.iterrows():         # 逐行遍历
    print(row['from'], row['to'])
```

三、sklearn 常用函数

数据分割

```
python

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

评估指标

```
python

from sklearn.metrics import (
    accuracy_score,      # 准确率 = (TP+TN) / 总数
    precision_score,     # 精确率 = TP / (TP+FP)
    recall_score,        # 召回率 = TP / (TP+FN)
    mean_squared_error,  # MSE
    confusion_matrix     # 混淆矩阵
)

accuracy_score(y_true, y_pred)
precision_score(y_true, y_pred, pos_label=1) # class 1 as positive
recall_score(y_true, y_pred, pos_label=0)    # class 0 as positive
```

模型

```
python

from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

lr = LogisticRegression()
lr.fit(X_train, y_train)
lr.predict(X_test)                # 预测标签
lr.predict_proba(X_test)[:, 1]   # 预测为class 1的概率（取第二列！）
```

四、IDS考试场景速查

Markov Chain

```
python
```

```

# 解hitting time / 吸收概率
A = np.array([[1-P[0,0], -P[0,2]],
              [-P[2,0], 1-P[2,2]]])
b = np.array([1.0, 1.0])          # 期望步数用[1,1]
# b = np.array([0, P_SM, 0])      # 概率类问题用对应概率
h = np.linalg.solve(A, b)

# 多步转移概率
P_n = np.linalg.matrix_power(P, n)
prob = P_n[i, j]

# Stationary distribution
eigvals, eigvecs = np.linalg.eig(P.T)
idx = np.argmin(np.abs(eigvals - 1))
pi = np.real(eigvecs[:, idx])
pi = pi / pi.sum()

```

分类评估 (threshold优化)

```

python

# 找cost最优threshold
best_threshold = 0
min_cost = float('inf')
for t in np.arange(0, 1.01, 0.01):
    cost = avg_cost_function(y_true, y_proba, t)
    if cost < min_cost:
        min_cost = cost
        best_threshold = t

# 找accuracy最优threshold
max_acc = 0
for t in np.arange(0, 1.01, 0.01):
    y_pred = (y_proba >= t).astype(int)
    acc = np.sum(y_pred == y_true) / len(y_true)
    if acc > max_acc:
        max_acc = acc
        best_threshold_acc = t

# 手写TP/FP/FN
y_pred = (y_proba >= threshold).astype(int)
TP = np.sum((y_pred == 1) & (y_true == 1))
FP = np.sum((y_pred == 1) & (y_true == 0))
FN = np.sum((y_pred == 0) & (y_true == 1))
TN = np.sum((y_pred == 0) & (y_true == 0))

```

SVD

```
python

U, d, Vt = np.linalg.svd(X, full_matrices=False)
D = np.diag(d)

# Explained variance
sv_sq = d ** 2
EV = np.array([np.sum(sv_sq[:k])/np.sum(sv_sq) for k in range(1, len(d)+1)])
k = np.argmax(EV >= 0.99) + 1 # 最小的k使EV>=0.99

# 低秩重构
X_approx = U[:, :k] @ np.diag(d[:k]) @ Vt[:k, :]

# 重构误差（每行）
error = np.linalg.norm(X - X_approx, axis=1)

# 找outlier（恰好100个）
threshold = np.sort(error)[::-1][99] # 第100大的值
outliers = X[error >= threshold]
```

Hoeffding置信区间

```
python
```

```

# 生成每个样本的cost数组
costs = []
for i in range(len(y_true)):
    if y_true[i]==1 and y_pred[i]==1:
        costs.append(80)    # TP
    elif y_true[i]==0 and y_pred[i]==1:
        costs.append(150)   # FP
    elif y_true[i]==1 and y_pred[i]==0:
        costs.append(900)   # FN
    else:
        costs.append(0)     # TN
costs = np.array(costs)

# Hoeffding公式
n = len(costs)
mean_cost = np.mean(costs)
a, b = 0, 900              # cost的上下界
delta = 0.05               # 95%置信

epsilon = (b-a) * np.sqrt(np.log(2/delta) / (2*n))
ci_lower = mean_cost - epsilon
ci_upper = mean_cost + epsilon

```

Markov Chain模拟 (simulation)

```
python
```

```

# 估计期望步数
n_sim = 10000
steps_list = []
for _ in range(n_sim):
    current = 0          # 起点
    steps = 0
    while current != target:
        current = np.random.choice(n_states, p=P[current])
        steps += 1
    steps_list.append(steps)
print(np.mean(steps_list))

# 估计吸收概率
count = 0
for _ in range(n_sim):
    current = start
    while current not in absorbing_states:
        current = np.random.choice(n_states, p=P[current])
    if current == target:
        count += 1
print(count / n_sim)

```

五、常见易错点

错误	原因	解决
<code>range(threshold)</code>	<code>range</code> 只接受整数	用 <code>np.arange()</code>
<code>y_proba >= threshold</code> 报错 shape错	<code>y_proba</code> 是2D的	取 <code>y_proba[:, 1]</code>
<code>list object is not callable</code>	<code>list</code> 被当变量名用了	重启kernel
<code>NameError: name 'u' is not defined</code>	写了 <code>u.np.xxx</code>	改成 <code>np.xxx</code>
<code>float('inf')</code> 忘了初始化	累加前没给初始值	在循环前加 <code>min_cost = float('inf')</code>
SVD的 <code>Vt</code> 不需要转置	<code>np.linalg.svd</code> 返回的已经是 V^T	直接用 <code>Vt[0,:]</code> 取第一个右奇异向量

错误	原因	解决
<code>full_matrices=False</code> 忘了加	<code>shape</code> 不符合题目要求	<code>np.linalg.svd(X,</code> <code>full_matrices=False)</code>